

API Documentation

Book Management API

Backend .NET Internship
Week 3 – Day 5

Prepared by: Mithgal Alhrene
Date: 6-8-2026

1. Introduction

This document describes the Book Management API based solely on the provided Postman Collection and execution screenshots. The API exposes REST endpoints for Create, Read, Update, and Delete (CRUD) operations using JSON payloads.

2. API Overview

Method	Endpoint	Purpose
GET	{{baseUrl}}/api/books	Retrieve all books
GET	{{baseUrl}}/api/books/{id}	Retrieve a book by ID
POST	{{baseUrl}}/api/books	Create a new book
PUT	{{baseUrl}}/api/books/{id}	Update an existing book
DELETE	{{baseUrl}}/api/books/{id}	Delete a book

3. Base URL & Environment

All requests use the Postman Environment variable {{baseUrl}}. Updating this variable allows the same collection to run against different environments without changing request URLs.

4. Endpoints

GET_ALL

Method: GET

URL: {{baseUrl}}/api/books

Purpose: Executes the GET ALL operation.

Request Body:

No request body required.

Success Response: 200 OK

HomeWorkspacesAPI Network

Search PostmanCtrl K

Invite

Upgrade

Mithgal Alhrene's WorkspaceNewImport

Collections

BinX-Training

Books

GETGET_ALL

GETGET_By_Id

GETGET_By_Id_NotFound

POSTPOST_Create

POSTPOST_Create_Invalid

PUTPUT_Update

PUTPUT_Update_NotFound

PUTPUT_Update_Invalid_Id

DELETEDELETE

DELETEDELETE_NotFound

Delivery Platform API

Environments

History

Flows

GETGET_ALL

BinX-Training / Books / GET_ALL

GET

{[baseUri]} /api/books

Send

Docs

Params

Authorization

Headers (6)

Body

Scripts

Settings

Cookies

Query Params

Key	Value	Description
Key	Value	Description

Body

Cookies

Headers (4)

Test Results

200 OK13 ms374 B

Save Response

JSON

Preview

Visualize

```
1 [
2   {
3     "bookId": 1,
4     "bookName": "Clean Code",
5     "authorId": 1,
6     "publishedYear": 2000,
7     "price": 50.00,
8     "author": null,
9     "reviews": []
10  },
11  {
12    "bookId": 2,
13    "bookName": "Refactoring",
14    "authorId": 2,
15    "publishedYear": 2010,
16    "price": 70.00,
17    "author": null,
18    "reviews": []
19  }
20 ]
```

Online

Find and replace

Console

Postbot

Runner

Start Proxy

Cookies

Vault

Trash

25°C

Search

ENG

42%

7:44 PM

8/6/2026

GET_By_Id

Method: GET

URL: `{{baseUrl}}/api/books/1`

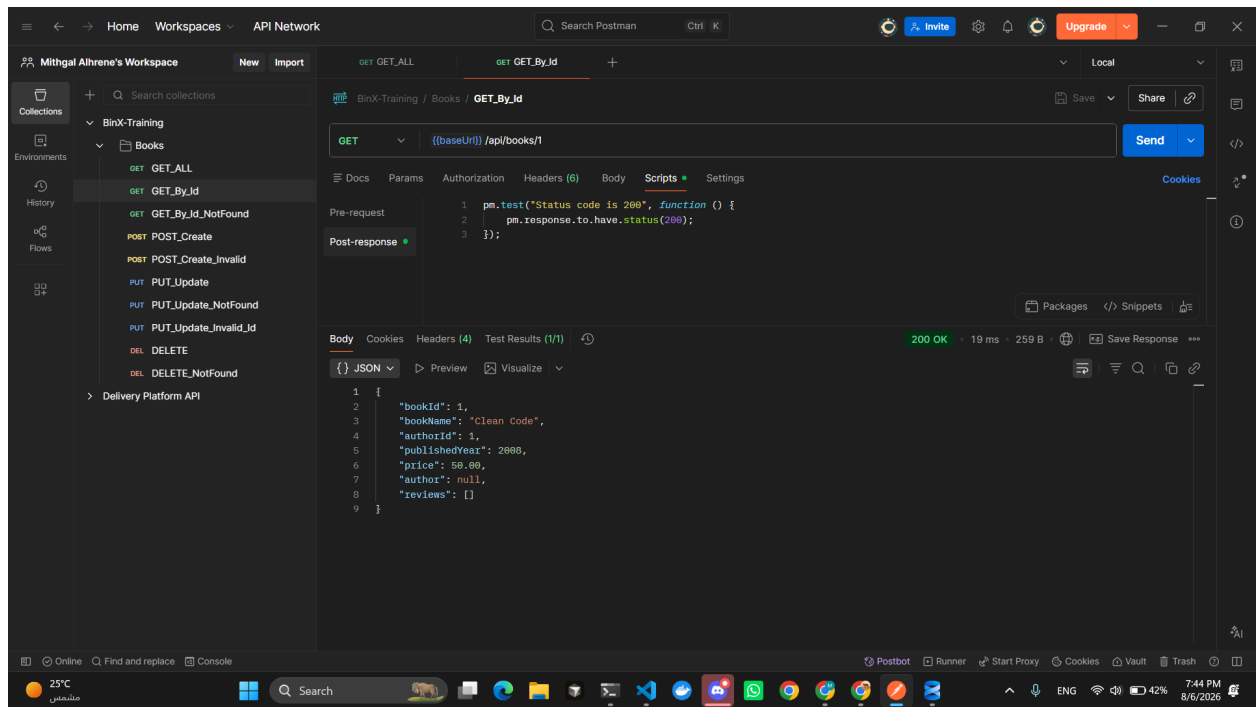
Purpose: Executes the GET By Id operation.

Request Body:

No request body required.

Success Response: 200 OK

Possible Error Response: 404 Not Found



GET_By_Id_NotFound

Method: GET

URL: {{baseUrl}}/api/books/99999

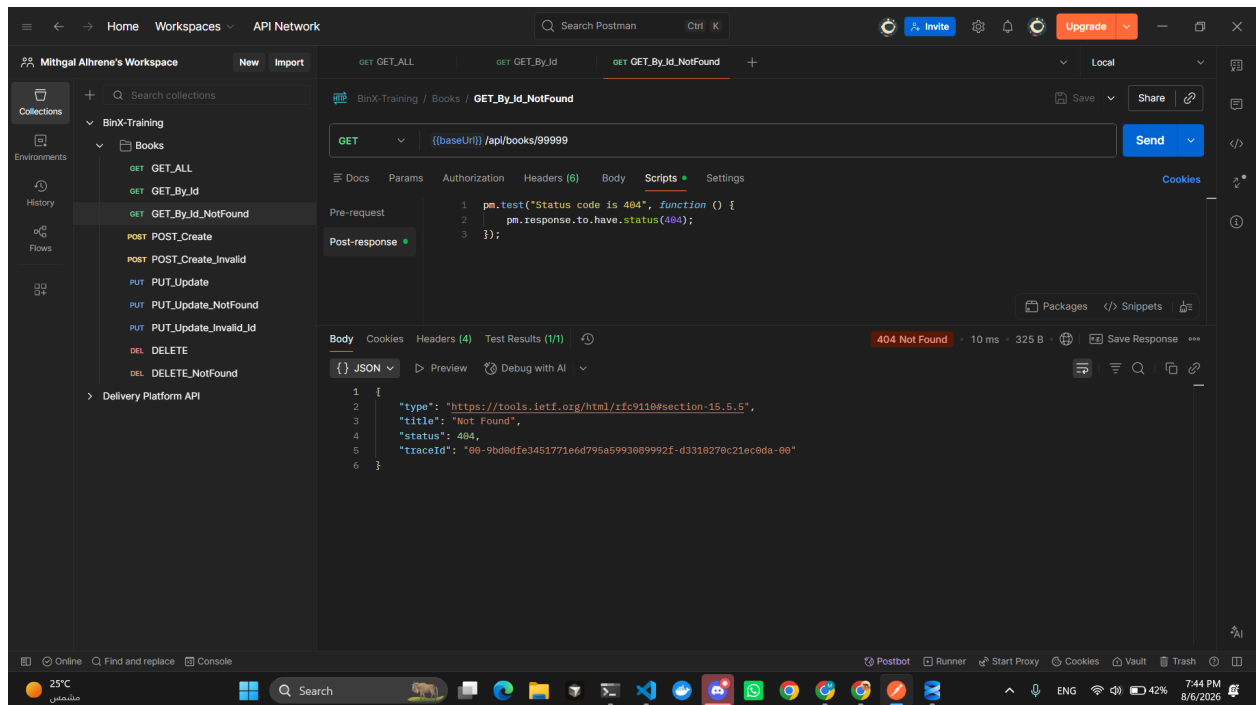
Purpose: Executes the GET By Id Not Found operation.

Request Body:

No request body required.

Success Response: 404 Not Found

Possible Error Response: 404 Not Found



POST_Create

Method: POST

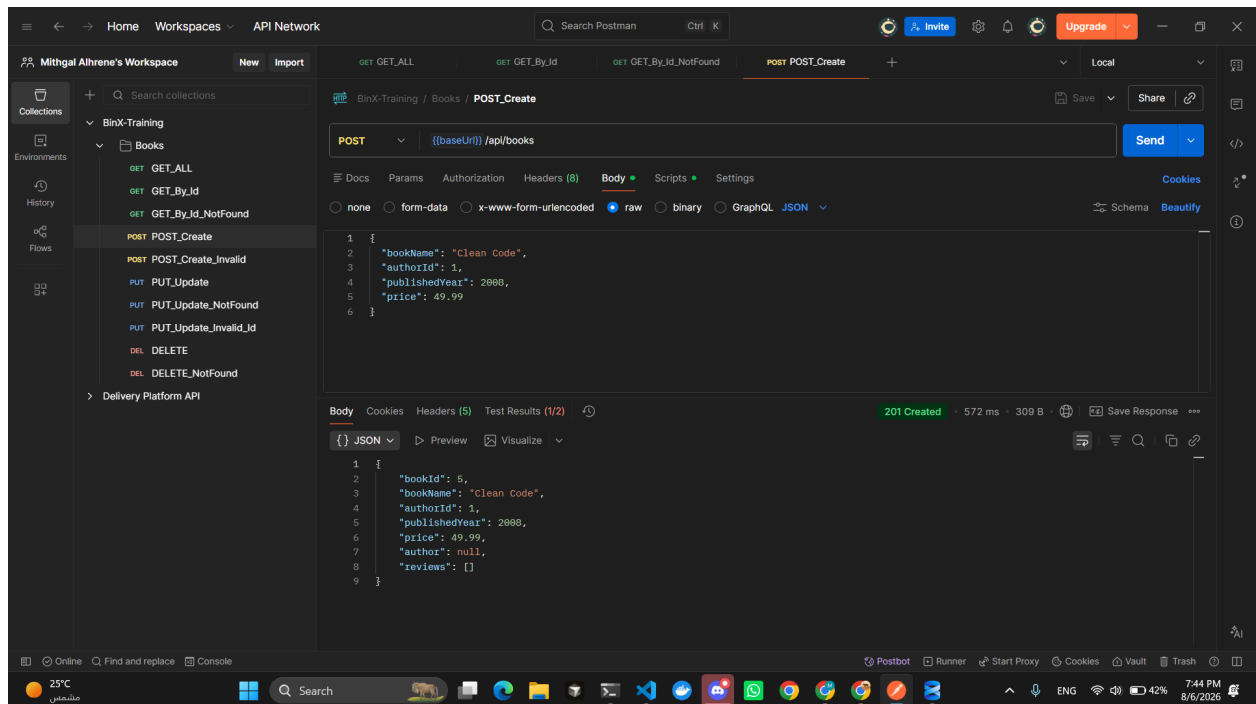
URL: `{{baseUrl}}/api/books`

Purpose: Executes the POST Create operation.

Request Body:

```
{ "bookName": "Clean  
Code", "authorId": 1, "publishedYear": 2008, "price": 49.99 }
```

Success Response: 201 Created



POST_Create_Invalid

Method: POST

URL: {{baseUrl}}/api/books

Purpose: Executes the POST Create Invalid operation.

Request Body:

```
{}
```

Success Response: 500 Internal Server Error

Possible Error Response: 500 Internal Server Error

The screenshot displays the Postman interface for a REST client. The left sidebar shows a collection named 'BinX-Training' with a sub-collection 'Books'. The 'POST POST_Create_Invalid' endpoint is selected. The URL is set to '[[baseUrl]]/api/books'. The request body is an empty JSON object '{}'. The response tab shows a '500 Internal Server Error' with a status of 500, a response time of 163 ms, and a size of 6.66 KB. The raw response body contains a detailed exception stack trace from Microsoft.EntityFrameworkCore, indicating a conflict with a FOREIGN KEY constraint in the 'FK_Books_Authors_AuthorId' table. The stack trace includes the following details:

- Exception: Microsoft.EntityFrameworkCore.UpdateException: An error occurred while saving the entity changes. See the inner exception for details.
- Inner Exception: Microsoft.Data.SqlClient.SqlException (0x80131904): The INSERT statement conflicted with the FOREIGN KEY constraint "FK_Books_Authors_AuthorId". The conflict occurred in database "HelloInXERCOREdb", table "dbo.Authors", column "AuthorId".
- Stack Trace: at Microsoft.Data.SqlClient.SqlConnection.OnError(SqlException exception, Boolean breakConnection, Action`1 wrapCloseInAction) at Microsoft.Data.SqlClient.SqlInternalConnection.OnError(SqlException exception, Boolean breakConnection, Action`1 wrapCloseInAction) at Microsoft.Data.SqlClient.TdsParser.ThrowExceptionAndWarning(TdsParserStateObject stateObj, SqlCommand command, Boolean callerHasConnectionLock, Boolean asyncClose) at Microsoft.Data.SqlClient.TdsParser.TryRun(RunBehavior runBehavior, SqlCommand cmdHandler, SqlDataReader dataStream, BulkCopySimpleResultSet bulkCopyHandler, TdsParserStateObject stateObj, Boolean& dataReady) at Microsoft.Data.SqlClient.SqlDataReader.TryReadInternal(Boolean timeout, Boolean& more) at Microsoft.Data.SqlClient.SqlDataReader.ReadAsyncExecute(Task task, Object state) at Microsoft.Data.SqlClient.SqlDataReader.InvokeAsyncCall[T](SqlDataReaderBaseAsyncCallContext`1 context) --- End of stack trace from previous location --- at Microsoft.EntityFrameworkCore.Update.AffectedCountModificationCommandBatch.ConsumeResultSetAsync(Int32 startCommandIndex, RelationalDataReader reader, CancellationToken cancellationToken) ClientConnectionId:dec8397-9c9f-4f48-9c13-d5144d5216ad Error Number:547,State:8,Class:16 --- End of inner exception stack trace ---

The bottom of the screenshot shows the Windows taskbar with various application icons and system tray information, including the date 8/6/2026 and time 7:45 PM.

PUT_Update

Method: PUT

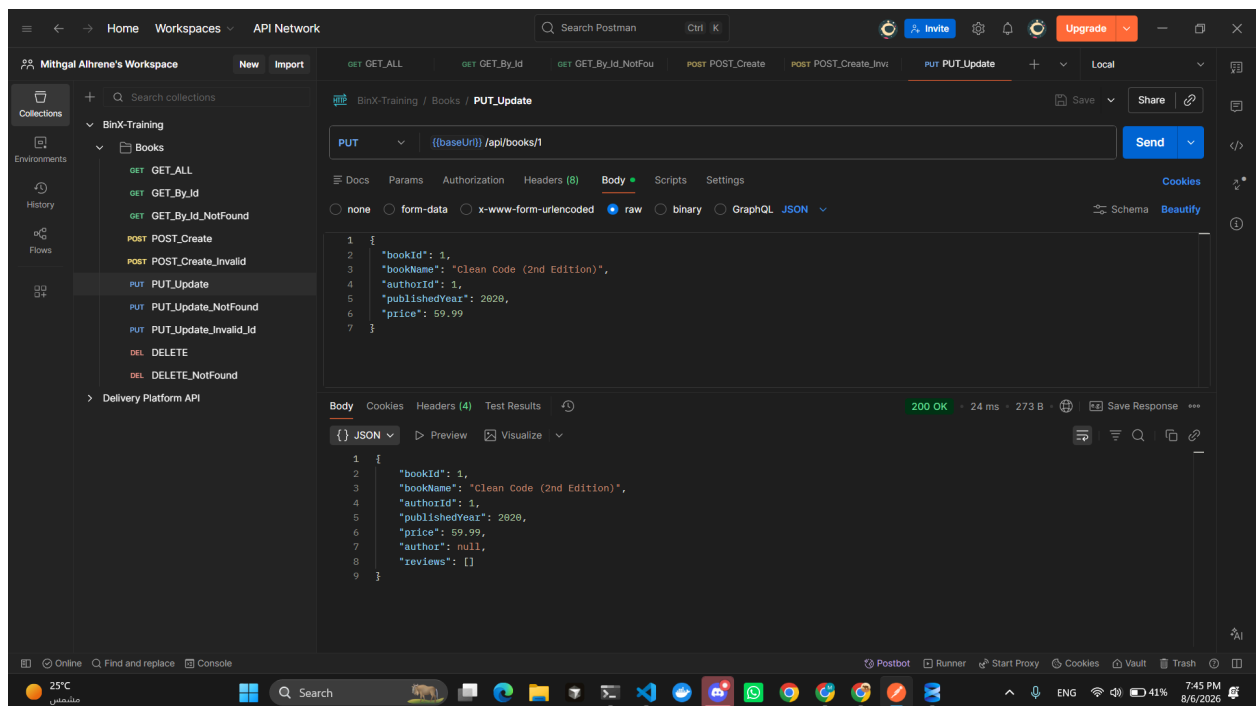
URL: `{{baseUrl}}/api/books/1`

Purpose: Executes the PUT Update operation.

Request Body:

```
{ "bookId":1,"bookName":"Clean Code (2nd Edition)","authorId":1,"publishedYear":2020,"price":59.99 }
```

Success Response: 200 OK



PUT_Update_NotFound

Method: PUT

URL: `{{baseUrl}}/api/books/99999`

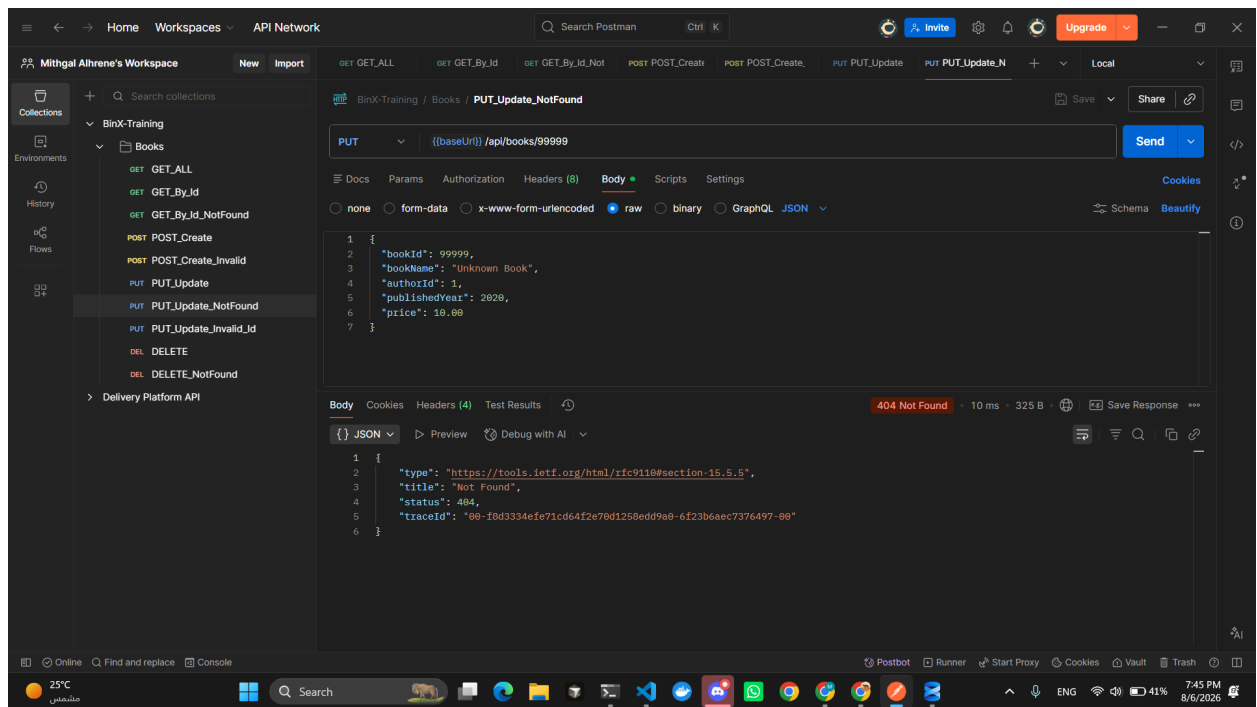
Purpose: Executes the PUT Update NotFound operation.

Request Body:

```
{ "bookId":99999,"bookName":"Unknown Book","authorId":1,"publishedYear":2020,"price":10.00 }
```

Success Response: 404 Not Found

Possible Error Response: 404 Not Found



PUT_Update_Invalid_Id

Method: PUT

URL: {{baseUrl}}/api/books/abc

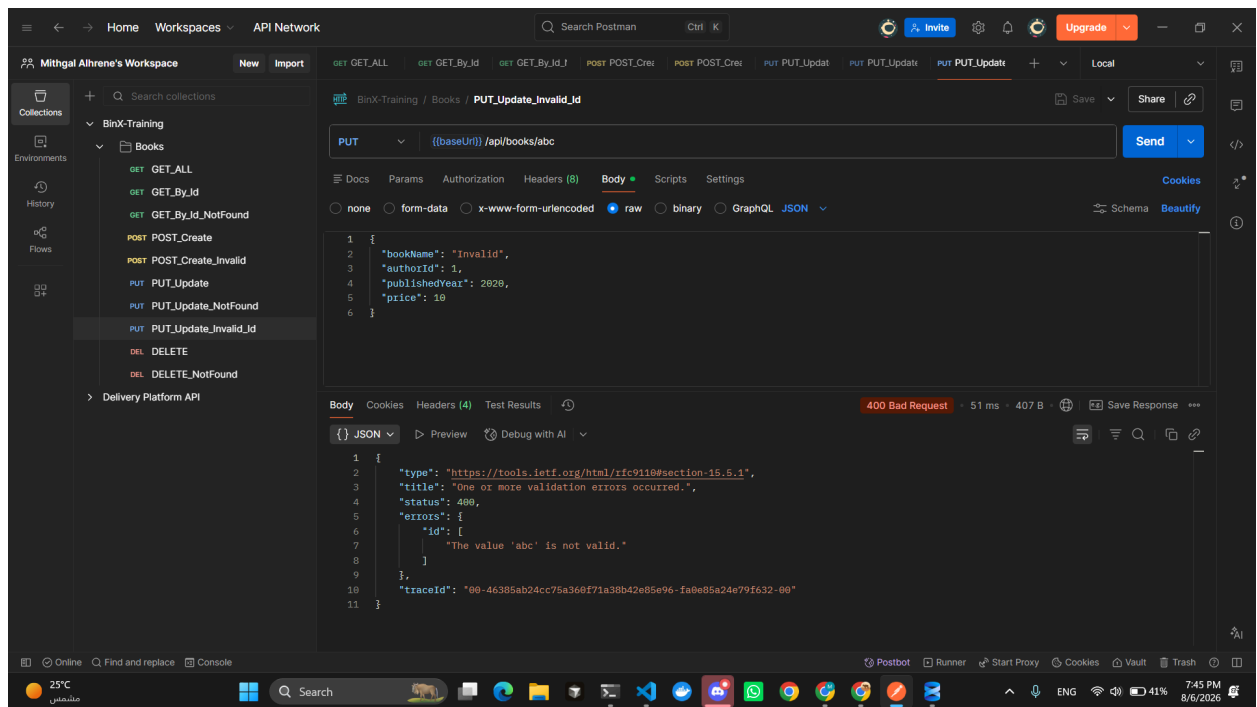
Purpose: Executes the PUT Update Invalid Id operation.

Request Body:

```
{ "bookName": "Invalid", "authorId": 1, "publishedYear": 2020, "price": 10 }
```

Success Response: 400 Bad Request

Possible Error Response: 400 Bad Request



DELETE

Method: DELETE

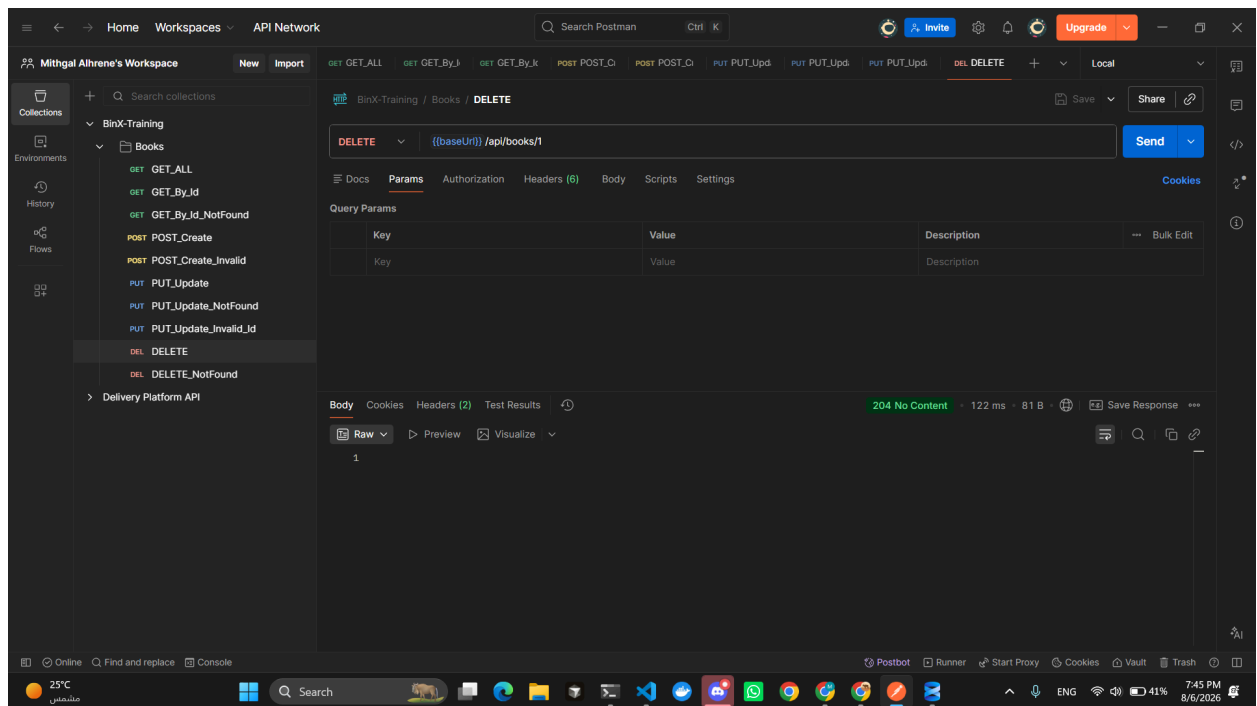
URL: `{{baseUrl}}/api/books/1`

Purpose: Executes the DELETE operation.

Request Body:

No request body required.

Success Response: 204 No Content



DELETE_NotFound

Method: DELETE

URL: {{baseUrl}}/api/books/99999

Purpose: Executes the DELETE NotFound operation.

Request Body:

No request body required.

Success Response: 404 Not Found

Possible Error Response: 404 Not Found

The screenshot shows the Postman interface with a workspace named 'Mithgal Alhrene's Workspace'. The 'API Network' tab is active, displaying a collection of API endpoints under 'BinX-Training / Books'. The selected endpoint is 'DELETE_NotFound' with a method of 'DELETE' and a URL of '{{baseUrl}}/api/books/99999'. The 'Params' tab is selected, showing a table with columns 'Key', 'Value', and 'Description'. The 'Body' tab is also visible, showing a JSON response with a status of '404 Not Found'.

Key	Value	Description
Key	Value	Description

```
1 {
2   "type": "https://tools.ietf.org/html/rfc9110#section-15.5.5",
3   "title": "Not Found",
4   "status": 404,
5   "traceId": "00-ec49d714243533cf4ff17aff0779552-45ec4c9d1b4aa348-00"
6 }
```

5. Happy Path Testing

Successful execution was verified for GET_ALL (200 OK), GET_By_Id (200 OK), POST_Create (201 Created), PUT_Update (200 OK), and DELETE (204 No Content).

6. Error Path Testing

Negative scenarios verified include GET_By_Id_NotFound (404), PUT_Update_NotFound (404), PUT_Update_Invalid_Id (400), DELETE_NotFound (404), and POST_Create_Invalid (500 Internal Server Error).

7. Postman Tests

The Postman Collection contains test scripts for GET_By_Id (200), GET_By_Id_NotFound (404), and POST_Create (201 with response ID verification), as defined in the provided collection.

8. Conclusion

The provided Postman Collection demonstrates CRUD functionality for the Book Management API. The included screenshots confirm both successful and error scenarios, making this document suitable for internship submission.